

Université Sorbonne Paris Nord
Ecole d'Ingénieurs Sup Galilée

TP5

Réalisé par:

BRAHIMI Kaoutar

Exercice 1 : Comprendre l'atomicité d'une transaction

Dans ce travail pratique, nous avons expérimenté la gestion des transactions dans un système de base de données à travers deux connexions distinctes, que nous avons nommées S1 et S2.

Désactivation de l'autocommit

Dès le départ, nous avons désactivé la validation automatique des transactions dans les deux sessions via la commande :

```
SET AUTOCOMMIT OFF;
```

Cela signifie que les opérations réalisées (insertion, mise à jour, suppression...) ne sont pas immédiatement sauvegardées, et peuvent donc être annulées tant qu'un **commit** n'a pas été effectué. Ce paramètre nous a permis d'étudier en profondeur le principe d'atomicité, l'une des propriétés clés des transactions.

Création de la table transaction (dans S1)

Dans la session S1, nous avons créé une table appelée transaction :

```
CREATE TABLE transaction (  
    idTransaction VARCHAR2(44),  
    valTransaction NUMBER(10)  
);
```

Manipulations dans la session S2

Dans S2, nous avons procédé à plusieurs opérations :

- Insertion :

```
INSERT INTO transaction VALUES ('T1', 120);
```

```
INSERT INTO transaction VALUES ('T2', 300);
```

- Modification :

```
UPDATE transaction SET valTransaction = 150 WHERE idTransaction = 'T2';
```

- Suppression :

```
DELETE FROM transaction WHERE idTransaction = 'T1';
```

Toutes ces actions ont ensuite été annulées par un simple :

`ROLLBACK;`

Résultat : La table est revenue à son état initial, comme si aucune opération n'avait eu lieu. Cela démontre clairement que tant qu'une transaction n'est pas validée, elle peut être entièrement annulée.

Fermeture sans validation (session S2)

Après un premier rollback, nous avons réinséré des lignes :

```
INSERT INTO transaction VALUES ('T1', 200);
```

```
INSERT INTO transaction VALUES ('T2', 450);
```

Mais cette fois, nous avons **quitté la session sans valider** : `QUIT;`

Depuis S1, nous avons constaté que les lignes insérées étaient absentes. Cela prouve que toute modification non validée est perdue lors de la fermeture d'une session, et reste invisible aux autres utilisateurs.

Fermeture brutale de S1

Toujours dans une optique de test, nous avons inséré des données dans S1 sans faire de COMMIT, puis nous avons brusquement fermé SQL*Plus.

À la reconnexion, il n'y avait aucune trace des changements.

Ajout de colonne + rollback

Dans une autre session, nous avons :

- Inséré de nouvelles lignes,
- Ajouté une colonne val2transaction avec :

```
ALTER TABLE transaction ADD val2transaction NUMBER(10);
```

Puis effectué : `ROLLBACK;`

Résultat : la table est revenue à son état d'origine, sans les nouvelles lignes ni la colonne ajoutée.

Concepts clés abordés

Session : Une session correspond à une connexion active entre l'utilisateur et la base de données. Elle permet d'exécuter des requêtes, de gérer des transactions et d'interagir avec les données de manière isolée par rapport aux autres sessions.

Transaction : Une transaction est un ensemble d'opérations SQL exécutées comme une unité indivisible. Grâce aux propriétés ACID :

- **Atomicité** : tout ou rien est appliqué.
- **Cohérence** : la base passe d'un état valide à un autre.
- **Isolation** : les transactions s'exécutent indépendamment.
- **Durabilité** : une fois validées, les modifications sont permanentes.

COMMIT : Applique définitivement les changements effectués depuis le début de la transaction ou depuis le dernier commit.

ROLLBACK : Annule tous les changements non validés, restaurant la base dans son état précédent.

Exercice n2 : Transactions concurrentes :

Contrairement à l'exercice précédent où on a systématiquement utilisé SET AUTOCOMMIT OFF, avant le début de chaque session, dans cet exercice, on laisse le réglage d'isolation par défaut.

Deux tables sont créées pour gérer les réservations de billets d'avion :

Création des deux tables vol et client et Insertion d'un vol et de deux clients :

```
CREATE TABLE vol ( idVol VARCHAR2(44),
```

```
capaciteVol NUMBER(10),
```

```
nbrPlacesReserveesVol NUMBER(10) );
```

```
CREATE TABLE client (
```

```
idClient VARCHAR2(44),
```

```
prenomClient VARCHAR2(11),
```

```
nbrPlacesReserveesClient NUMBER(10) );
```

```
INSERT INTO vol VALUES ('V1', 100, 0);
```

```
INSERT INTO client VALUES ('C1', 'Alice', 0);
```

```
INSERT INTO client VALUES ('C2', 'Bob', 0);
```

- Transaction T1 : réserve des billets pour C1 (mise à jour sur vol et client).

- Transaction T2 : travaille en parallèle sans voir les modifications non validées par T1.

Nous observons :

La transaction T1 peut voir immédiatement ses propres modifications avant leur validation (*commit*). Par exemple, si T1 effectue une réservation de places, elle pourra consulter ces changements lors de ses lectures ultérieures, même s'ils ne sont pas encore confirmés dans la base de données.

En revanche, ces modifications restent invisibles pour les autres transactions concurrentes (comme T2) tant que T1 n'a pas été validée. Cette isolation garantit une gestion sécurisée des transactions en évitant :

- Les conflits entre opérations concurrentes.
- Les anomalies liées à la lecture de données intermédiaires non validées.

Cela illustre la propriété d'isolation des transactions en mode READ COMMITTED.

COMMIT et ROLLBACK :

Maintenant, nous allons effectuer un ROLLBACK de la transaction en cours (T1), puis valider les mises à jour avec un COMMIT. Nous observerons également le comportement de la transaction T2 par rapport à ces actions.

ROLLBACK de la transaction T1 :

Après avoir exécuté la commande ROLLBACK, les modifications effectuées par la transaction T1

Seront annulées et la base de données reviendra à son état initial. Toutes les opérations non validées seront abandonnées.

Nous observons :

- La base revient à son état initial.
- Exécuter un COMMIT ou un ROLLBACK n'a plus aucun effet, car nous sommes au début d'une nouvelle transaction. Dans cette nouvelle transaction, les effets de la précédente transaction ont été annulés.
- Pour la transaction T2, tout se passe comme si la transaction T1 n'avait pas existé.
- Les modifications effectuées par la transaction T1 ne sont pas visibles pour la transaction T2, car elles n'ont pas été validées.

Validation des mises à jour avec COMMIT :

START TRANSACTION;

```
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 1 WHERE idClient = 'C1';
```

```
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 1 WHERE idVol = 'V1';
```

```
COMMIT;
```

Effets du COMMIT :

- Lorsque T1 est validée (COMMIT), ses modifications deviennent permanentes
- Ces changements ne peuvent plus être annulés (principe ACID de durabilité)
- La base de données garantit que ces modifications survivront même en cas de panne système

Observations :

1. Visibilité des modifications :
 - o Après validation, T2 voit immédiatement les changements de T1
 - o Ce comportement correspond au niveau READ COMMITTED (par défaut dans Oracle)
 - o Dans ce mode :
 - § Chaque transaction ne voit que les données validées par les autres
 - § Les modifications en cours restent invisibles
2. Comparaison avec REPEATABLE READ :
 - o Dans ce niveau plus strict :
 - § T2 aurait conservé une vue figée de la base depuis son démarrage
 - § Les modifications de T1 seraient restées invisibles pour T2
 - § Nécessiterait un nouveau démarrage de T2 pour voir les changements.
 - §

Isolation incomplète = incohérence possible

Toujours en mode READ COMMITTED. Maintenant nous allons dérouler deux transactions T1

(Réservation de 2 billets pour c1) et T2 (réservation de 3 billets pour c2) en parallèle selon l'alternance suivante :

- on effectue les deux sélections pour T1 (vol et client)
- on effectue les deux sélections pour T2 (vol et client)
- on effectue les deux mises à jour de T1, et on valide
- on effectue les deux mises à jour de T2, et on valide

```
START TRANSACTION;
```

```
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 1 WHERE idClient = 'C1';
```

```
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 1 WHERE idVol = 'V1';
```

```
COMMIT;
```

```
START TRANSACTION;
```

```
SELECT * FROM vol WHERE idVol = 'V1' FOR UPDATE;
```

```
SELECT * FROM client WHERE idClient = 'C2' FOR UPDATE;
```

```
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 3 WHERE idVol = 'V1';
```

```
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 3 WHERE idClient = 'C2';
```

```
COMMIT;
```

Après l'exécution de ces transactions, il apparaît que les deux clients ont réservé un total de 5 billets (2 pour le client C1 et 3 pour le client C2), tandis que seulement 3 billets ont été enregistrés pour le vol. Cette situation engendre une incohérence au sein de la base de données : bien que chaque transaction soit individuellement correcte, l'ensemble aboutit à un état incohérent, en raison d'un niveau d'isolation insuffisant (READ COMMITTED).

Isolation complète = blocage et rejet des transactions possibles

Afin de garantir une cohérence totale et d'éviter les incohérences liées aux transactions concurrentes, il est indispensable d'adopter le mode d'isolation complet (serializable). Ce mode assure un niveau maximal d'isolation en bloquant ou en rejetant les transactions susceptibles de générer des anomalies.

Après avoir réinitialisé les deux sessions en sélectionnant le mode serializable, nous reproduisons l'exécution concurrente précédente. Cette fois, l'une des deux transactions sera automatiquement rejetée, préservant ainsi l'intégrité de la base de données.

Reprenons maintenant l'exemple d'exécution concurrente vu en cours :

Session T1 :

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
START TRANSACTION;
```

```
SELECT * FROM vol WHERE idVol = 'V1' FOR UPDATE;
```

```
SELECT * FROM client WHERE idClient = 'C1' FOR UPDATE;
```

```
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 2 WHERE idVol = 'V1';
```

```
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 2 WHERE idClient = 'C1';
```

```
COMMIT;
```

Session T2 :

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
START TRANSACTION;
```

```
SELECT * FROM vol WHERE idVol = 'V1' FOR UPDATE;
```

```
SELECT * FROM client WHERE idClient = 'C2' FOR UPDATE;
```

```
UPDATE vol SET nbrPlacesReservesVol = nbrPlacesReservesVol + 3 WHERE idVol = 'V1';
```

```
UPDATE client SET nbrPlacesReservesClient = nbrPlacesReservesClient + 3 WHERE idClient = 'C2';
```

```
COMMIT;
```

Dans ce scénario, l'une des deux transactions sera rejetée pour éviter les incohérences dans la base de données.

Cet exercice a permis d'explorer deux aspects fondamentaux des bases de données relationnelles : l'atomicité des transactions et le contrôle de concurrence. À travers des manipulations pratiques, nous avons validé les principes ACID :

Atomicité et durabilité :

- ü Les commandes COMMIT (validation permanente) et ROLLBACK (annulation) garantissent que les transactions sont traitées comme des unités indivisibles.
- ü Les fermetures de session non contrôlées annulent automatiquement les modifications non validées, préservant ainsi l'intégrité des données.

Isolation et concurrence :

- ü En mode READ COMMITTED (par défaut), nous avons observé des anomalies de concurrence (mises à jour perdues, incohérences) lorsque deux transactions modifiaient les mêmes données simultanément.
- ü Le passage en isolation SERIALIZABLE a résolu ces problèmes en bloquant ou rejetant les transactions conflictuelles, au prix d'une réduction du parallélisme.

Bilan des apprentissages :

- Le choix du niveau d'isolation est un compromis entre cohérence (isolation stricte) et performance (isolation faible).
- Les verrous explicites (FOR UPDATE) et les transactions courtes sont des bonnes pratiques pour minimiser les conflits.
- Oracle (comme d'autres SGBD) offre des mécanismes robustes pour garantir l'intégrité des données malgré la concurrence.